Guilherme Henrique Bernardi

Gustavo Franco Pereira

Victor Hugo Camargo

Manutenção Predial

Limeira – SP

2026

SUMÁRIO

1. INTRODUÇÃO 1
2. OBJETIVO 2
2.1. Objetivos Específicos 2
3. RESULTADOS ESPERADOS 3
4. LEVANTAMENTO DE REQUISITOS 4
4.1. Requisitos Funcionais 4
4.2. Requisitos Não Funcionais 4
4.3. Priorização 5
5. METODOLOGIA (SCRUM) 7
5.1. Por que utilizar o Scrum no projeto "PredialFix"? 7
6. DIAGRAMAS 9
6.1. Diagrama de Classe 9
6.2. Diagrama de Sequência 10
7. PROTOTIPAGEM 11
8. BANCO DE DADOS 14

1. INTRODUÇÃO

A manutenção predial eficiente é um dos pilares fundamentais para o pleno funcionamento de instituições educacionais de grande porte. No cenário atual do SENAI, a gestão da infraestrutura enfrenta um desafio crítico: a alta demanda por reparos, que variam de intervenções simples, como a troca de lâmpadas, a manutenções hidráulicas complexas, esbarra em um gargalo de comunicação lenta e falta de rastreabilidade. Atualmente, o fluxo de centenas de solicitações mensais carece de transparência, resultando em insatisfação entre funcionários e alunos, que não possuem visibilidade sobre o status de seus pedidos.

O projeto PredialFix surge como a resposta tecnológica a esse desafio. Focado no desenvolvimento de uma infraestrutura de Back-End robusta, o sistema visa transformar o fluxo desorganizado de solicitações em um processo de trabalho otimizado. Através de uma API segura e um servidor centralizado, o PredialFix oferecerá transparência total, permitindo que cada chamado seja acompanhado em tempo real, desde a abertura até a resolução definitiva. Mais do que um simples registro, a plataforma atuará como o cérebro da manutenção, sendo capaz de priorizar urgências e garantir que a infraestrutura da instituição suporte, com qualidade, as atividades de ensino e trabalho.

2. OBJETIVO

Desenvolver o Back-End da plataforma PredialFix, focado em uma infraestrutura de servidor robusta e em uma API segura que centralize a gestão de manutenção predial do SENAI, garantindo transparência total desde a abertura do chamado até a sua resolução final.

2.1. Objetivos Específicos

- **Gestão de Acesso e Segurança:** Sistema de autenticação seguro que permite a identificação de funcionários e alunos, garantindo a integridade dos dados e o registro de quem abriu cada solicitação.
- **Abertura e Categorização de Chamados:** Interface para descrever o problema (hidráulico, elétrico, estrutural), anexar evidências e localizar o incidente (bloco/sala).
- **Fluxo de Trabalho Transparente (Workflow):** Lógica de estados do chamado (Aberto, Em Análise, Em Execução, Concluído) que permite ao usuário acompanhar cada etapa em tempo real, eliminando a falta de feedback.
- **Algoritmo de Priorização de Urgências:** Classificação automática da criticidade dos pedidos, vazamentos graves têm prioridade sobre lâmpadas queimadas, organizando a fila de trabalho da equipe de manutenção de forma eficiente.
- **Histórico e Relatórios de Performance:** Armazenamento completo de dados para que a gestão visualize o tempo médio de atendimento e os problemas mais frequentes, facilitando decisões estratégicas de infraestrutura.

3. RESULTADOS ESPERADOS

A implementação do PredialFix deverá transformar a gestão de manutenção predial do SENAI, substituindo o atual modelo por um sistema preditivo e de alta visibilidade. Espera-se que o projeto proporcione uma redução drástica no tempo de resposta aos incidentes, uma vez que a infraestrutura de Back-End permitirá a triagem e priorização automatizada das demandas. A transparência será o resultado mais perceptível para a comunidade escolar; ao permitir que alunos e funcionários acompanhem em tempo real cada etapa do processo, da abertura à resolução, a plataforma eliminará a incerteza e as reclamações por falta de feedback. Por fim, a segurança garantida pela API autenticada assegurará a integridade dos dados, gerando um histórico confiável que servirá de base para auditorias e decisões estratégicas da diretoria.

4. LEVANTAMENTO DE REQUISITOS

A definição dos requisitos do PredialFix foca em transformar as necessidades operacionais do SENAI em uma infraestrutura de Back-End lógica e funcional. Este levantamento é o que garante que o sistema não seja apenas um canal de registros, mas uma ferramenta ativa de gestão que elimina falhas de comunicação e organiza o fluxo de trabalho da manutenção.

4.1. Requisitos Funcionais

1. RF01 - Autenticação e Gestão de Perfis: O sistema deve possuir um módulo de autenticação seguro que diferencie, no mínimo, dois níveis de acesso: Usuário Comum (funcionários/alunos que apenas abrem e acompanham seus chamados) e Responsável/Técnico (equipe de manutenção/chefes que atualizam e gerenciam os chamados).

2. RF02 - Abertura de Chamados e Priorização: O sistema deve disponibilizar um endpoint para registro de novos chamados, sendo obrigatório o envio das seguintes informações: Tipo de problema (Elétrica, Hidráulica, Outros), Descrição detalhada da ocorrência, Localização exata e Nível de Urgência.

3. RF03 - Máquina de Estados (Workflow): O sistema deve gerenciar o ciclo de vida do chamado, permitindo a transição estrita entre os seguintes status: *Aberto* ➔ *Em Análise* ➔ *Em Execução* ➔ *Concluído*.

4. RF04 - Filtragem e Listagem: O sistema deve permitir que os usuários filtrem e visualizem a lista de chamados com base em seus status atuais, localização e nível de urgência.

5. RF05 - Regra de Negócio de Atualização: O sistema deve bloquear a alteração de status de um chamado (ex: mover para "Em Execução") caso o usuário logado não tenha o perfil de "Responsável" ou não seja o chefe do setor correspondente.
6. RF06 - Histórico de Manutenção por Unidade: O sistema deve armazenar e permitir a consulta de todo o histórico de serviços já realizados, filtráveis por um espaço específico ou área comum (ex: "Laboratório de Informática 3"), para facilitar a análise de reincidências.
7. RF07 - Transparência e Notificações (Feedback): O sistema deve disparar atualizações de progresso para o criador do chamado a cada mudança de status (ex: "Em Análise", "Técnico a caminho", "Serviço Concluído"), garantindo a total transparência exigida pelo Senai.

4.2. Requisitos Não Funcionais

1. RNF01 - (Segurança e Autenticação): A comunicação com a API deve ser criptografada via HTTPS. A autenticação das requisições deve ser feita utilizando tokens seguros (como o padrão JWT - JSON Web Tokens), garantindo que apenas usuários logados acessem os dados.

2. RNF02 - (Desempenho): O servidor back-end deve ser capaz de processar as requisições de consulta e abertura de chamados com um tempo de resposta médio inferior a 2 segundos, mesmo lidando com as centenas de solicitações mensais da unidade.

3. RNF03 - (Disponibilidade): O sistema deve operar com alta disponibilidade (meta de *uptime* de 99%), garantindo que problemas estruturais possam ser relatados por alunos e funcionários a qualquer momento de funcionamento do Senai.

4. RNF04 - (Arquitetura e Integração): O back-end deve ser construído seguindo os padrões de uma API RESTful (ou arquitetura semelhante), retornando dados estruturados em formato JSON, para permitir a futura e fácil integração com o aplicativo mobile do PredialFix.

5. RNF05 - (Confiabilidade / Integridade de Dados): O sistema deve utilizar um banco de dados relacional (ex: PostgreSQL, MySQL) robusto, garantindo a integridade referencial entre os chamados, os usuários e os locais do prédio, sem perda de dados em caso de falhas nas transações.

6. RNF06 - (Documentação): A API desenvolvida deve ser documentada de forma clara (utilizando ferramentas como *Swagger/OpenAPI*), descrevendo rotas, parâmetros e respostas esperadas, facilitando o trabalho da equipe que for desenvolver a interface (Front-End/App).

4.3. Priorização

Para priorizar os requisitos do PredialFix de forma estratégica, foi utilizado o método MoSCoW. Essa técnica ajuda a focar no que é vital para o funcionamento da API e o que pode ser deixado para fases posteriores.

- MUST HAVE (DEVE TER)

São as funcionalidades inegociáveis para que o sistema exista. Sem elas, o projeto não atende ao objetivo principal de organizar os chamados.

- SHOULD HAVE (DEVERIA TER)

Funcionalidades de alta prioridade que devem ser incluídas, mas cuja ausência não impede o sistema de funcionar em curto prazo.

- COULD HAVE (PODERIA TER)

Recursos que agregariam valor e melhorariam a experiência do usuário, mas que só serão desenvolvidos se houver tempo e recursos sobrando.

- WON´T HAVE (NÃO TERÁ)

Funcionalidades que foram discutidas, mas que estão fora do escopo desta entrega inicial.

REQUISITO	PRIORIZAÇÃO	EXPLICAÇÃO
RF01, RF02, RF03	**Must have**	Essenciais para a funcionalidade básica do sistema: autenticação, abertura de chamados e gerenciamento do fluxo de trabalho.
RNF01, RNF05	**Must have**	Críticos para segurança dos dados e integridade das informações no sistema.
RF05, RF07	**Must have**	Garantem regras de negócio essenciais e transparência no acompanhamento dos chamados.
RF04, RF06	**Should have**	Importantes para facilitar a gestão e consulta de chamados, permitindo análise de reincidências.
RNF02, RNF03	**Should have**	Desempenho e disponibilidade são importantes para a experiência do usuário e confiabilidade do sistema.
RNF04, RNF06	**Could have**	Facilitam integração futura e manutenção, mas não são bloqueadores para o funcionamento inicial.

5. METODOLOGIA (SCRUM)

O Scrum é um framework (ou estrutura de trabalho) ágil focado na gestão e no desenvolvimento de projetos complexos. Em vez de tentar construir todo o sistema de uma só vez e entregá-lo apenas no final de meses de trabalho, o Scrum divide o projeto em ciclos curtos e contínuos chamados Sprints (que geralmente duram de 1 a 4 semanas).

Ao final de cada Sprint, a equipe entrega uma parte do produto que já funciona e agrega valor. Ele é baseado em três pilares fundamentais: Transparência, Inspeção e Adaptação, e organiza a equipe através de papéis bem definidos (Product Owner, Scrum Master e Desenvolvedores) e cerimônias rápidas (Planejamento, Reunião Diária, Revisão e Retrospectiva).

5.1. Por que utilizar o Scrum no projeto "PredialFix"?

O cenário do SENAI exige uma solução rápida, robusta e que resolva o problema da "falta de transparência e demora no atendimento". O Scrum é a escolha ideal para esse desafio pelos seguintes motivos:

- Entregas Incrementais e Rápidas (Sprints): Sua equipe não precisa entregar a API inteira de uma vez. Vocês podem usar as Sprints para entregar as *Funcionalidades Essenciais* em partes. Por exemplo: a Sprint 1 pode focar apenas na "Gestão de Usuários" (Autenticação no Laravel). A Sprint 2 foca na "Abertura de Chamados" e no banco de dados MySQL. Isso permite que o SENAI comece a testar o núcleo do sistema muito mais rápido.

- Priorização Inteligente (Product Backlog): O desafio fala em "priorizar urgências". No Scrum, os requisitos (as 5 funcionalidades essenciais) formam o *Product Backlog*, uma lista ordenada. O que é mais crítico para o SENAI resolver hoje? Provavelmente o registro do problema e o Workflow. O Scrum garante que a equipe foque primeiro no que traz mais impacto para os funcionários e alunos.

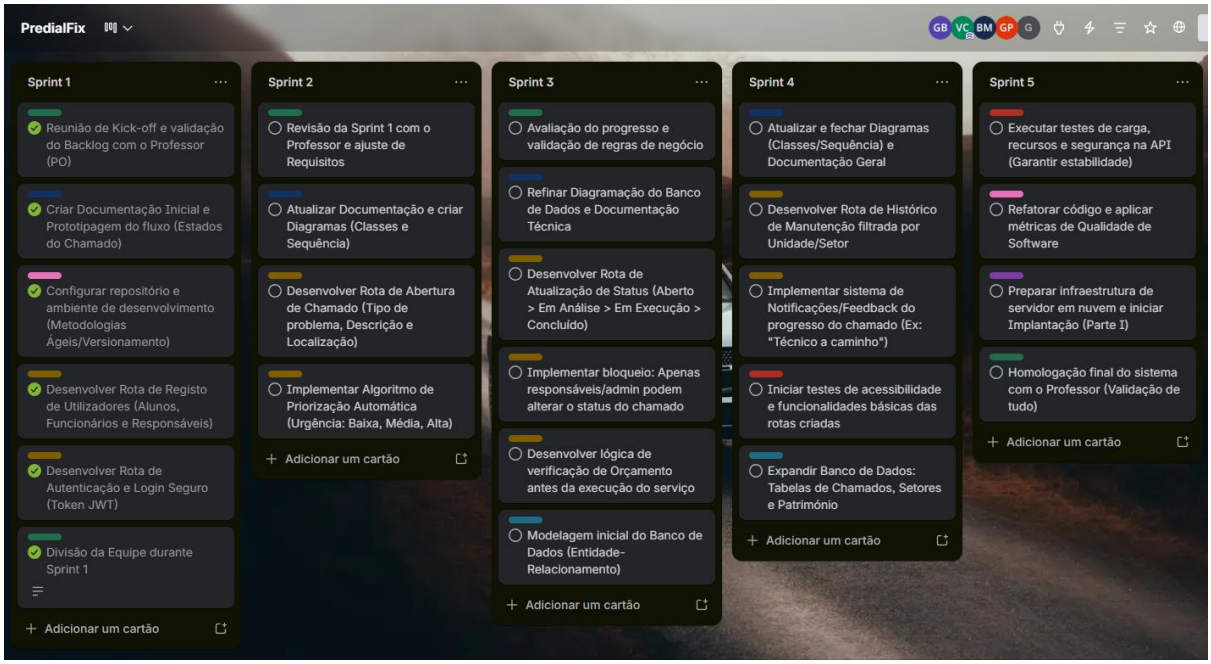
- Transparência e Alinhamento Constante: O problema central do SENAI é a falta de transparência nos chamados. Ironicamente, o Scrum resolve a falta de transparência no *desenvolvimento*. Com as reuniões diárias (Daily Scrum), a equipe de Back-End saberá exatamente quem está cuidando da validação de dados (Request Validation), quem está criando as rotas RESTful e quem está travado em algum problema com o Eloquent ORM.

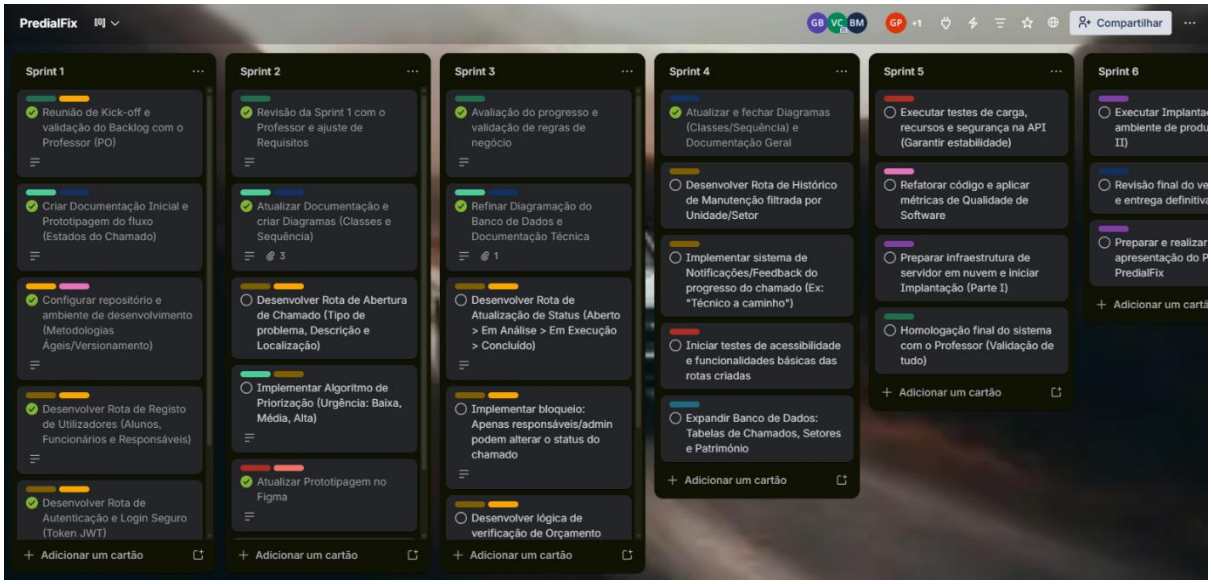
- Adaptação às Regras de Negócio: Modelar os status dos chamados (Aberto -> Em Análise -> Em Execução -> Concluído) e a lógica de quem pode atualizar o quê (multi-nível) pode gerar dúvidas durante o desenvolvimento. O Scrum permite que, ao final de cada Sprint (na Revisão), a equipe valide com os interessados se a API está se comportando como esperado, permitindo ajustes rápidos antes de avançar para as "Notificações de Progresso" ou "Histórico".

- Foco na Qualidade Técnica: Como há requisitos rigorosos (validação, respostas em JSON, documentação clara), as Sprints permitem que a equipe estabeleça uma "Definição de Pronto" (Definition of Done). Uma funcionalidade só avança se a rota estiver criada, testada, validada e com o código devidamente subido no GitHub.

Ao usar o Scrum, o desenvolvimento do PredialFix deixa de ser uma caixa preta demorada e passa a ser um processo previsível, focado em resolver a dor real do SENAI etapa por etapa.

Foi utilizado o Trello para a parte de metodologia: **Trello**


PredialFix
Sprint 1
Reunião de Kick-off e validação do Backlog com o Professor (PO)
Criar Documentação Inicial e Prototipagem do fluxo (Estados do Chamado)
Configurar repositório e ambiente de desenvolvimento (Metodologias Ágeis/Versionamento)
Desenvolver Rota de Registo de Utilizadores (Alunos, Funcionários e Responsáveis)
Desenvolver Rota de Autenticação e Login Seguro (Token JWT)
Divisão da Equipe durante Sprint 1
Adicionar um cartão
Sprint 2
Revisão da Sprint 1 com o Professor e ajuste de Requisitos
Atualizar Documentação e criar Diagramas (Classes e Sequência)
Desenvolver Rota de Abertura de Chamado (Tipo de problema, Descrição e Localização)
Implementar Algoritmo de Priorização Automática (Urgência: Baixa, Média, Alta)
Adicionar um cartão
Sprint 3
Avaliação do progresso e validação de regras de negócio
Refinar Diagramação do Banco de Dados e Documentação Técnica
Desenvolver Rota de Atualização de Status (Aberto > Em Análise > Em Execução > Concluído)
Implementar bloqueio: Apenas responsáveis/admin podem alterar o status do chamado
Desenvolver lógica de verificação de Orçamento antes da execução do serviço
Modelagem inicial do Banco de Dados (Entidade-Relacionamento)
Adicionar um cartão
Sprint 4
Atualizar e fechar Diagramas (Classes/Sequência) e Documentação Geral
Desenvolver Rota de Histórico de Manutenção filtrada por Unidade/Setor
Implementar sistema de Notificações/Feedback do progresso do chamado (Ex: "Técnico a caminho")
Iniciar testes de acessibilidade e funcionalidades básicas das rotas criadas
Expandir Banco de Dados: Tabelas de Chamados, Setores e Património
Adicionar um cartão
Sprint 5
Executar testes de carga, recursos e segurança na API (Garantir estabilidade)
Refatorar código e aplicar métricas de Qualidade de Software
Preparar infraestrutura de servidor em nuvem e iniciar Implantação (Parte I)
Homologação final do sistema com o Professor (Validação de tudo)
Adicionar um cartão

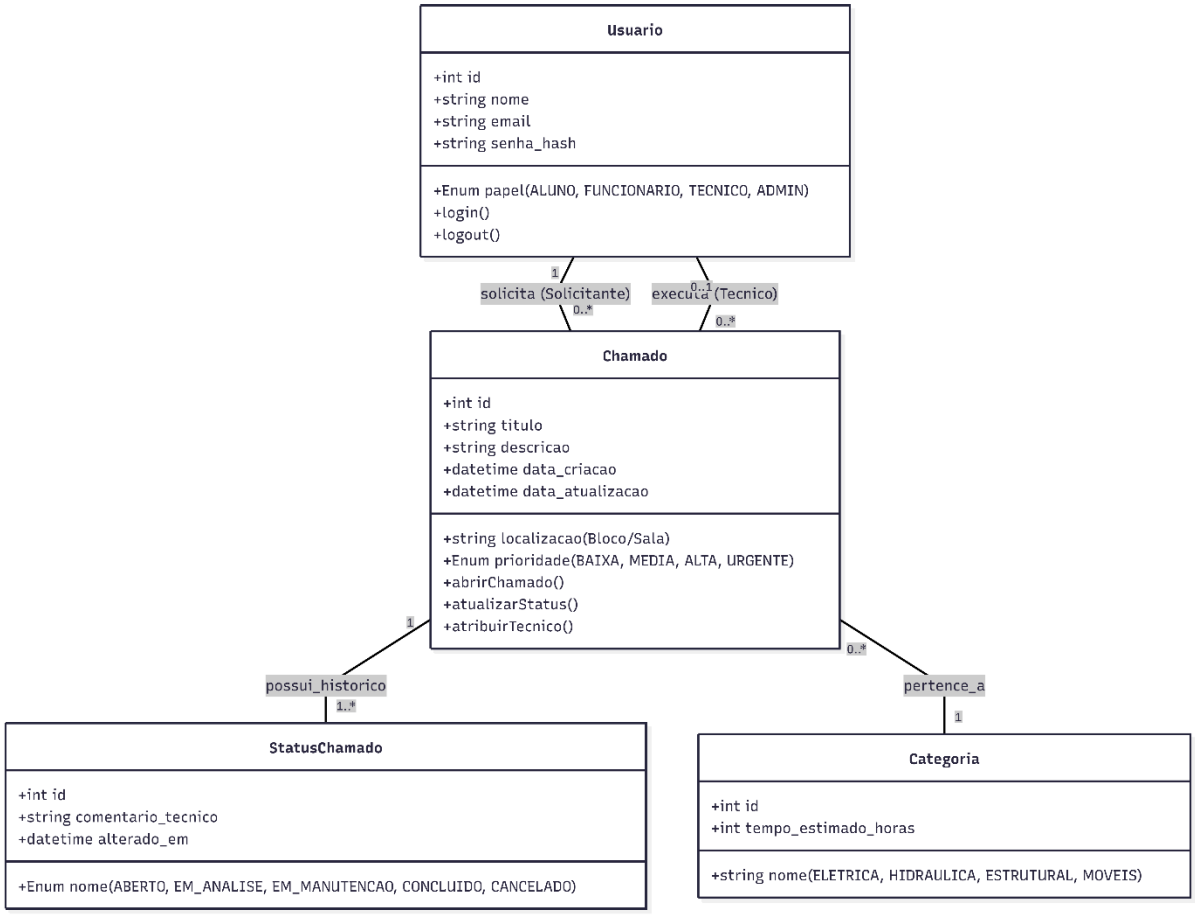

PredialFix
Compartilhar
Sprint 1
Reunião de Kick-off e validação do Backlog com o Professor (PO)
Criar Documentação Inicial e Prototipagem do fluxo (Estados do Chamado)
Configurar repositório e ambiente de desenvolvimento (Metodologias Ágeis/Versionamento)
Desenvolver Rota de Registo de Utilizadores (Alunos, Funcionários e Responsáveis)
Desenvolver Rota de Autenticação e Login Seguro (Token JWT)
Adicionar um cartão
Sprint 2
Revisão da Sprint 1 com o Professor e ajuste de Requisitos
Atualizar Documentação e criar Diagramas (Classes e Sequência)
3
Desenvolver Rota de Abertura de Chamado (Tipo de problema, Descrição e Localização)
Implementar Algoritmo de Priorização (Urgência: Baixa, Média, Alta)
Atualizar Prototipagem no Figma
Adicionar um cartão
Sprint 3
Avaliação do progresso e validação de regras de negócio
Refinar Diagramação do Banco de Dados e Documentação Técnica
1
Desenvolver Rota de Atualização de Status (Aberto > Em Análise > Em Execução > Concluído)
Implementar bloqueio: Apenas responsáveis/admin podem alterar o status do chamado
Desenvolver lógica de verificação de Orçamento
Adicionar um cartão
Sprint 4
Atualizar e fechar Diagramas (Classes/Sequência) e Documentação Geral
Desenvolver Rota de Histórico de Manutenção filtrada por Unidade/Setor
Implementar sistema de Notificações/Feedback do progresso do chamado (Ex: "Técnico a caminho")
Iniciar testes de acessibilidade e funcionalidades básicas das rotas criadas
Expandir Banco de Dados: Tabelas de Chamados, Setores e Património
Adicionar um cartão
Sprint 5
Executar testes de carga, recursos e segurança na API (Garantir estabilidade)
Refatorar código e aplicar métricas de Qualidade de Software
Preparar infraestrutura de servidor em nuvem e iniciar Implantação (Parte I)
Homologação final do sistema com o Professor (Validação de tudo)
Adicionar um cartão
Sprint 6

6. DIAGRAMAS

Os diagramas são as representações visuais que estruturam a lógica do sistema, funcionando como uma ponte entre os requisitos de negócio e a implementação técnica. Eles permitem que a arquitetura da API seja compreendida de forma clara, garantindo que o fluxo de informações entre o usuário e o servidor ocorra de maneira segura e organizada.

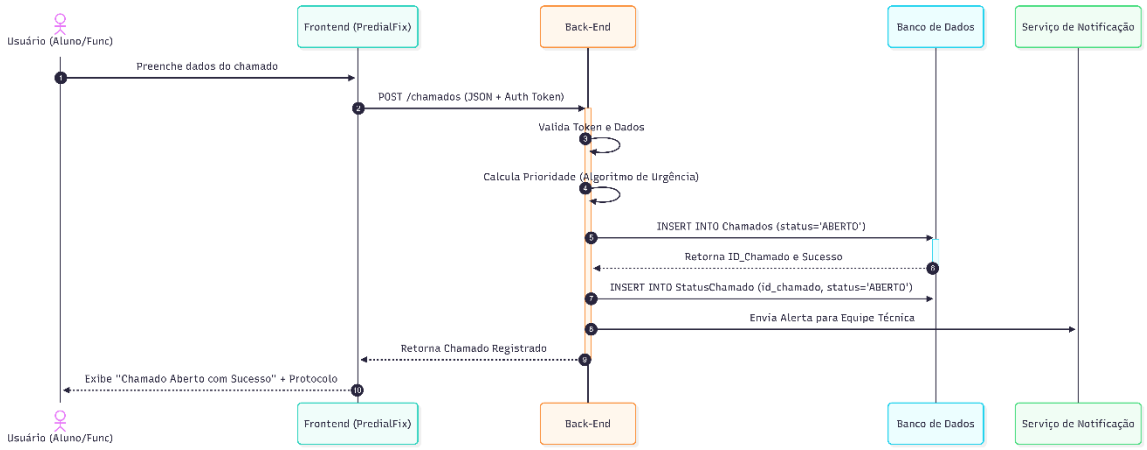
6.1. Diagrama de Classe

Mapeia a estrutura estática dos dados, definindo as entidades principais, seus atributos e as relações de cardinalidade. É essencial para garantir a integridade do banco de dados e a organização das informações.



6.2. Diagrama de Sequência

Descreve o comportamento dinâmico do sistema, ilustrando a ordem cronológica das interações entre o usuário, a API e o banco de dados. Validando fluxos críticos, como a abertura de chamados, garante que as regras de segurança e priorização sejam aplicadas antes da persistência dos dados.



7. PROTOTIPAGEM

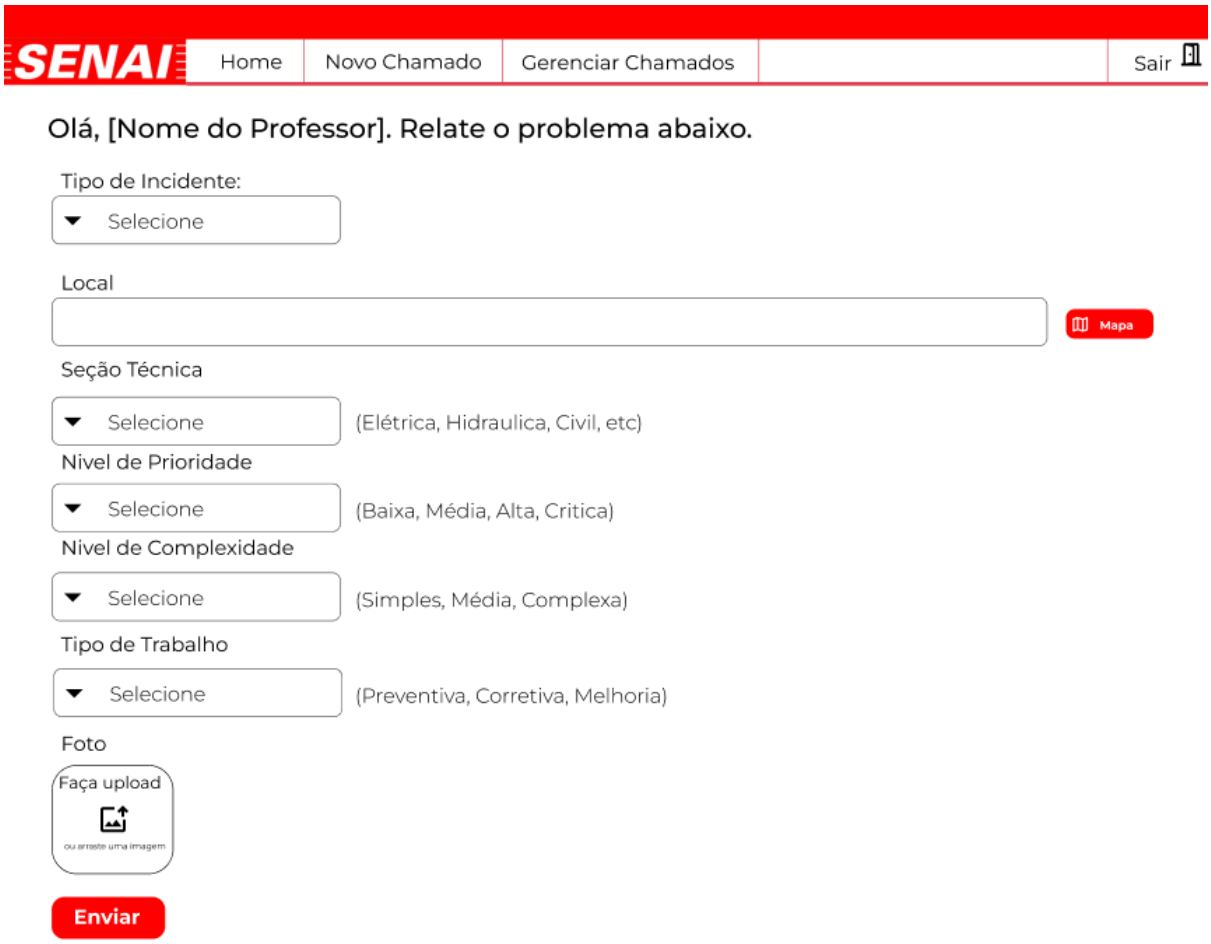
Foi utilizado o Figma para o desenvolvimento e prototipagem do aplicativo: **Figma**

Login

Menu

SENAI
Home
Novo Chamado
Gerenciar Chamados
Sair
Chamados Feitos
32
Chamados em Andamento
32
Chamados Feitos
32
Chamados Feitos
32
Chamados Recentes
Tipo
Descrição
Local
Data de Abertura
Status
Data de Término
Elétrica
Tomada em Curto...
Bloco A, Sala 1
02/01/2026
Em Andamento
02/01/2026
Elétrica
Tomada em Curto...
Bloco A, Sala 1
02/01/2026
Em Andamento
02/01/2026
Hidraulica
Tomada em Curto...
Bloco A, Sala 1
02/01/2026
Em Andamento
02/01/2026
Elétrica
Tomada em Curto...
Bloco A, Sala 1
02/01/2026
Em Andamento
02/01/2026
Relatar novo Problema

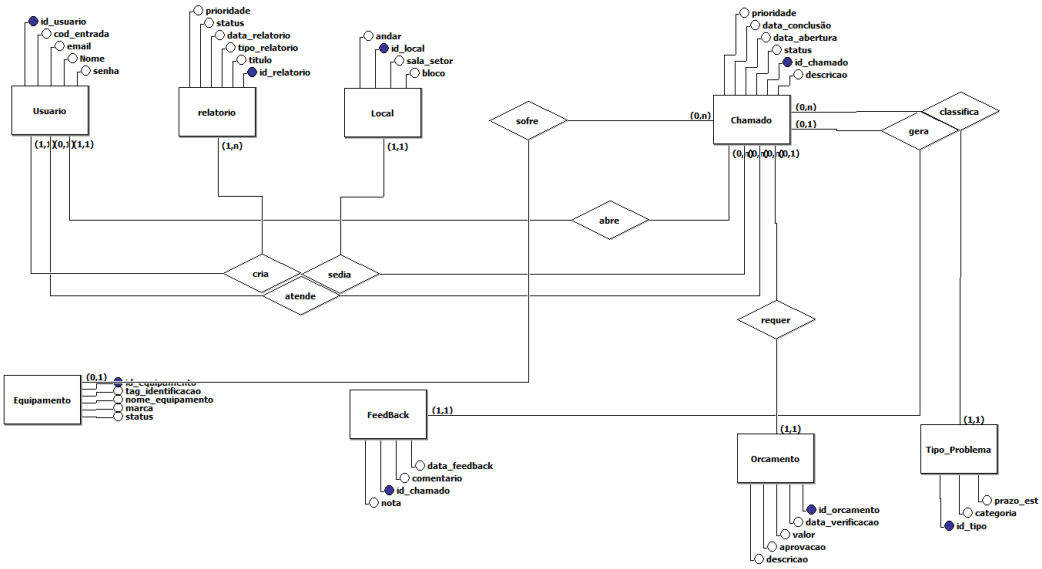
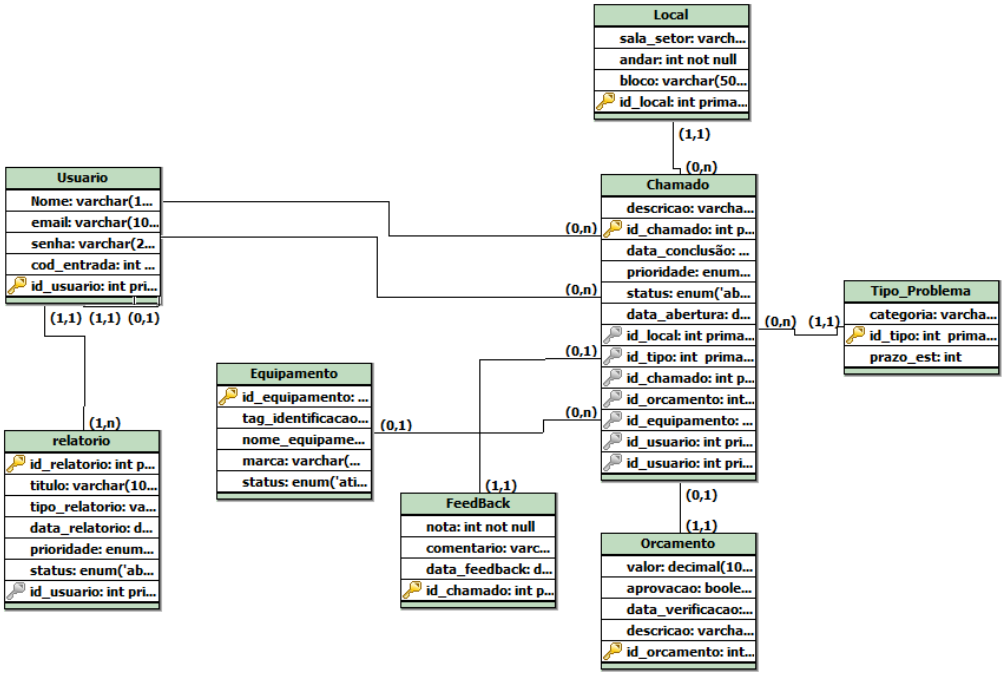
Registrar Chamado



8. BANCO DE DADOS

Foi utilizado o BrModelo para a modelagem do banco de dados do aplicativo.

Modelo Conceitual



Modelo Lógico